

VI History for LabVIEW VIs in VS Code Git Repos

Using Magic-Byte Detection

Executive summary

This report specifies how to build (and how to modify the attached implementation to complete) a Visual Studio Code extension that adds a context-menu command **“VI History”** when a user right-clicks a file in a Git repository, **shown only when that file is a LabVIEW VI by content** (magic bytes) **and has at least two commits that modified it**. VI detection must **not** rely on filename or extension; instead the extension must read **4 ASCII bytes at offset 8 (0-based)** and accept `LVIN` or `LVCC`. Evidence for this signature appears in LabVIEW file-format research and community notes: the RSRC container header is followed by the “file type” field (e.g., `LVIN` / `LVCC`) and related signatures in early bytes. ¹

The required UX behavior (“show only if eligible”) is best implemented using VS Code’s `menu when clauses` plus a dynamically maintained **setContext-backed membership map** (`resourcePath in <contextKey>`) as supported by VS Code’s `in` operator. ² The extension should index eligible files in the background (bounded, cached, and debounced for performance), and expose a webview-based history viewer with actions such as **Open at commit**, **Diff vs previous**, **Copy hash**, and **Generate LVCompare HTML report**.

The attached repository `/mnt/data/vi-history-suite/vi-history-suite-main` already implements most of the required core: Git tracked-file enumeration via `git ls-files -z`, bounded commit query via `git log -n 2 --format=%H --follow -- <path>`, magic-byte VI detection by partial reads, and a webview “VI History” panel. Key gaps relative to the requested spec are mainly: (a) adopting the exact context key names/when-clause formulation requested, (b) implementing **LVCompare / LabVIEWCLI report generation** with the mandated filename scheme `{type}-report-{fullFilename}.html`, (c) verifying **both revisions’ blobs are VIs** before running LVCompare/CLI, and (d) implementing robust **LabVIEW 2026 Q1 32-bit/64-bit detection and LVCompare selection** at runtime.

All guidance is current as of **2026-04-02 (America/Hermosillo)** and prioritizes official VS Code docs, Git documentation, and NI documentation as requested.

Repository inspection and mapping to requirements

What exists already in the attached repo

Based on inspection of the uploaded zip extracted to `/mnt/data/vi-history-suite/vi-history-suite-main` (local inspection; no external citations):

The extension is already a Node/TypeScript VS Code extension with:

- A contributed Explorer context menu item `VI History` (`contributes.menus.explorer/context`) gated by a **membership context key** and workspace trust.
- Magic-byte detection modules:
- `src/domain/viMagicCore.ts` defines signatures and includes a strict option to also check the `RSRC` header.
- `src/domain/viFile.ts` and `src/domain/viMagic.ts` implement *partial* reads for `file://` URIs and a fallback to `workspace.fs.readFile` for others.
- Git CLI integration in `src/git/gitCli.ts`:
- `git ls-files -z` for tracked files
- `git log -n <limit> --format=%H --follow -- <path>` for commit list
- `git rev-parse --show-toplevel` and `git rev-parse HEAD`
- Eligibility indexing in `src/indexing/viEligibilityIndexer.ts`:
- Runs on activation, iterates tracked files, checks magic bytes, checks commit count ≥ 2 , then pushes eligible absolute paths into a context key object.
- Uses `p-limit` to cap concurrency.
- Uses `window.withProgress` for a notification progress experience.
- History viewer in `src/ui/historyPanel.ts` and command handler in `src/commands/openViHistoryCommand.ts`:
- Loads commit list, supports “open at commit” (via `toGitUri`) and “diff vs previous” (via `vscode.diff`).

What is missing or misaligned with the requested spec

The key deltas to reach the exact user spec:

- **Context key names and when clause:** the request specifies a `when` clause like `resourcePath in labviewViHistory.eligiblePaths && isWorkspaceTrusted && gitOpenRepositoryCount >= 1`. The repo currently uses `viHistorySuite.eligiblePaths` and `viHistorySuite.isWorkspaceTrusted` (custom). VS Code already exposes `isWorkspaceTrusted` for `when` clauses and provides the Workspace Trust API; prefer using the built-in key for compliance and simplicity. ³
- **Editor context menus:** the repo contributes only to `explorer/context`. The request asks to cover `editor/title/context` (optional) and mentions `editor/title` and `editor/context` locations. VS Code supports these in `contributes.menus`. ⁴
- **Report generation:** no implementation exists for generating LVCompare/LabVIEWCLI reports; must be added, including:
 - Mandatory filename format: `{type}-report-{fullFilename}.html`.
 - Verification of blob magic bytes for both revisions before invoking the compare tool.
 - Storage strategy (`context.storageUri` recommended) and secure webview linking (via `asWebviewUri` and `localResourceRoots`). ⁵
- **LabVIEW 2026 Q1 32/64 adaptation:** repo has settings placeholders but no robust runtime detection. Must implement:
- Windows registry / install-path probing and “preferBitness” logic.

- Cross-platform fallbacks (macOS, Linux) acknowledging current NI platform constraints (e.g., macOS Community Edition notes and differing support timelines). ⁶
- **Progress UX:** notification progress exists; request also wants a **progress bar design** with percent/items/ETA and surfaced through:
 - `window.withProgress` updates (increment/message)
 - a discreet **Status Bar progress item** (recommended for background progress) and
 - optional webview progress messaging. ⁷

VS Code extension APIs and UX primitives to use

Menus, commands, when clauses, and `setContext` membership maps

Menu contributions are declared in `package.json` under `contributes.menus`. VS Code supports menu locations including `explorer/context`, `editor/context`, `editor/title`, and `editor/title/context`. ⁸

A critical enabling pattern is to compute eligibility at runtime and maintain a context key (via the built-in `setContext` command) whose value is an object used with the `in` operator in a menu `when` clause. VS Code explicitly documents `in` / `not in` plus `setContext` for dynamic menu gating. ⁹

Key details: - `resourcePath` is a context key that represents the active resource path and can be used for matching. ¹⁰ - VS Code includes a `gitOpenRepositoryCount` context key; you can use comparisons like `gitOpenRepositoryCount >= 1`. ¹¹ - Workspace Trust can be used in UI gating using the built-in `isWorkspaceTrusted` context key. ¹² - Commands invoked from a context menu are passed the selected resource URI when VS Code can infer it (Explorer passes URI of selected resource; editor passes URI of document). ¹³

Enablement vs visibility: VS Code notes that `when` applies to menus while `enablement` applies to commands broadly. If you want “visible only if eligible,” prefer `when` gating the menu item; also keep command handler resilient (show a message if invoked without eligibility). ¹⁴

Activation events and the `workspaceContains` caveat

The extension should avoid eager activation (`*`) and instead activate using a combination that supports background indexing without slowing startup:

- `onStartupFinished` triggers after VS Code startup and is designed not to slow startup. ¹⁵
- `workspaceContains:<glob>` activates when an opened folder contains at least one file matching a glob pattern. ¹⁶

Caveat: `workspaceContains` is based on matching a **file** glob; relying on `.git/` (a directory) may be unreliable. This is why many extensions use `onStartupFinished` plus runtime checks (e.g., `gitOpenRepositoryCount >= 1` in menus) rather than trying to “pre-detect git repositories” via `workspaceContains`. ¹⁷

WebViewPanel, localResourceRoots, storage, and security

Using a **WebViewPanel** is appropriate for the history viewer because it enables a table/timeline UI beyond native tree constraints. VS Code's WebView guide highlights:

- Use `WebView.asWebViewUri` to load local resources securely. ¹⁸
- Restrict file access via `WebViewOptions.localResourceRoots` (least privilege). ¹⁹
- Follow webview security practices: minimal capabilities (`enableScripts` only when needed) and a CSP. ²⁰

For storing generated HTML comparison reports, use `ExtensionContext.storageUri` (workspace-specific directory) which is recommended for larger workspace-scoped files. ²¹

TreeView and TimelineProvider considerations

A **TreeView** can render a simplified history list in the sidebar as an alternative UI. This is entirely stable and documented as a common workbench UI primitive. However, the request also mentions **TimelineProvider**.

As of the referenced sources, **TimelineProvider** is still a proposed API (`vscode.proposed.timeline.d.ts` exists), meaning it is **not publishable** without enabling proposed APIs (Insiders + `--enable-proposed-api`), and proposed APIs cannot be used in published Marketplace extensions. ²²

Practical conclusion: implement the webview (and optionally a **TreeView**) in the published extension; treat **TimelineProvider** as an experimental branch behind a build flag for internal use only.

Workspace.fs vs Node fs, virtual workspaces, and trust

File IO and Git operations differ across environments:

- `workspace.fs.readFile(uri)` reads the **entire contents** of a file as bytes; there is no range API, so it's inefficient for probing just 12 bytes. ²³
- For `file://` URIs on desktop/remote extension hosts, Node's `fs.open` / `fs.read` can read only the first 12 bytes efficiently.
- For non-file schemes (virtual workspaces, custom file systems), you may have no path on disk; you must fall back to `workspace.fs.readFile`, accepting the cost.

Additionally, Workspace Trust should gate any tool execution (Git CLI, LVCompare, LabVIEWCLI) because these execute external binaries. VS Code provides static declarations via `capabilities.untrustedWorkspaces` and a runtime API `workspace.isTrusted`. ²⁴

Git integration and eligibility algorithm

Git integration approaches and tradeoffs

The extension needs: - list tracked files - determine commit count (≥ 2) that modified a specific file, preferably following renames - retrieve blobs at specific commits for verification and for opening/diffing - handle multiple repositories (multi-root + submodules)

Below is a comparison table focused on VS Code extensions:

Approach	How it works	Pros	Cons	Complexity
VS Code built-in Git extension API (<code>vscode.git</code>)	Activate <code>vscode.git</code> , call <code>getAPI(1)</code> , use <code>repositories</code> , <code>Repository.log({path,maxEntries})</code> , <code>Repository.buffer(ref,path)</code> , <code>toGitUri()</code>	No need to ship Git logic; integrates with VS Code's repo discovery; <code>toGitUri</code> supports diff/open workflows; can read blob bytes via <code>buffer(ref,path)</code>	API isn't part of core <code>vscode</code> namespace; must depend on <code>vscode.git</code> and handle it being disabled; still may depend on external Git	Medium
Git CLI (spawn <code>git</code>)	Call <code>git ls-files -z</code> , <code>git log -n 2 --format=%H --follow -- <path></code> , <code>git show <hash>:<path></code> , <code>git rev-parse</code>	Most deterministic and well documented; available everywhere Git is installed; easy to bound by <code>-n 2</code> ; <code>--follow</code> handles renames best-effort	Requires <code>git</code> in PATH (or discover it); process overhead; must handle quoting and Windows peculiarities	Medium
libgit2 / NodeGit	Native bindings to libgit2	High performance; no <code>git</code> dependency	Native module complexity, platform builds, ABI issues; packaging pain for VS Code Marketplace	High
isomorphic-git	Pure JS Git implementation	Works where <code>child_process</code> is restricted (some environments)	Not feature-complete vs native Git; can be slower; still needs filesystem access	High

Given the prompt's prioritization of official docs and Git man pages, the best pragmatic implementation is **hybrid**: 1) Attempt to use `vscode.git` API for repo discovery, blob reads (`Repository.buffer`) and

`toGitUri` diff/open. ²⁷

2) Use **Git CLI** as a reliable fallback (especially for `--follow` semantics, which may not be identical in all APIs). ²⁸

Required Git commands and why

Tracked-file enumeration should use:

- `git ls-files -z` (tracked files; NUL-terminated, no quoting), ideal for safe parsing of unusual filenames. ²⁹

Eligibility commit count should use a bounded query:

- `git log -n 2 --format=%H --follow -- <path>`
- `-n 2` returns at most 2 commits, so runtime is bounded.
- `--follow` continues history across renames, but Git documents that it works only for a **single file**. ²⁸

Blob verification and report generation inputs should use:

- `git show <hash>:<path>` to retrieve the blob contents: Git's `git show` documentation states that for plain blobs, it shows the plain contents. ³⁰
Combine with `gitrevisions` syntax (`<rev>:<path>`) rules if you need to support richer revision specs. ³¹

Repo root and HEAD hash:

- `git rev-parse --show-toplevel` to anchor paths and determine repo root. ³²
- `git rev-parse HEAD` to cache eligibility per HEAD. (Git plumbing command; same man page.) ³²

Eligibility algorithm specification

The requested behavior: show "VI History" only when the right-clicked file is: 1) tracked in Git 2) a LabVIEW VI by magic bytes (`LVIN` or `LVCC` at offset 8) 3) modified by at least two commits

Recommended algorithm (large-repo safe):

1) Determine active repositories: - Use `vscode.git` API `repositories` list when available. ³³
- Fallback: attempt `git rev-parse --show-toplevel` in each workspace folder.

2) For each repository: - Enumerate tracked files: `git ls-files -z`. ²⁹
- For each tracked path, compute absolute fs path and perform magic-byte check (see next section). - If magic-byte passes, run bounded commit query: - `git log -n 2 --format=%H --follow -- <relPath>` - Eligible iff number of hashes returned ≥ 2 . ²⁸

3) Cache and invalidate: - Cache key: `(repoRoot, relPath, HEAD)`; if HEAD unchanged, reuse previous eligibility result. - Invalidate on: - `vscode.git` repository events when available (`onDidOpenRepository`, `onDidCloseRepository`, and ideally repository state change). ³⁴ - filesystem changes to `.git/HEAD` or `.git/index` if you need CLI-only mode.

4) Concurrency and debouncing: - Use a concurrency cap (e.g., 4–16 depending on system).
- Debounce multiple triggers (repo open/close, trust changes) into a single reindex run.

5) Best-effort rename handling: - `--follow` is best-effort and documented as single-file only. ³⁵
- Document that eligibility and history across complex rename histories may be incomplete (e.g., copies or detection failures).

VI detection by magic bytes at offset 8

Evidence for LVIN/LVCC signatures and offset

LabVIEW VIs are typically RSRC container files. Community and research sources consistently describe the early header structure as:

- A resource header begins with `RSRC` and a version; then a “file type” field such as `LVIN` (VI), `LVCC` (class), or related values appears in early bytes. ¹
- NI Community discussion about VI “magic number” references `LVINLBW` and notes structure details immediately before it. ³⁶
- PRONOM research issue for LabVIEW VIs aligns with these signatures in its format identification effort. ³⁷

Given the user’s explicit requirement, the extension’s VI detection must:

- Read 4 bytes at offset 8 (0-based) and interpret as ASCII
- Accept only `LVIN` or `LVCC`
- Not rely on extension; allow any filename

Optionally, to reduce false positives, a “strict” mode may also validate `RSRC` at offset 0. Ryan Pacini’s file format write-up describes RSRC header bytes at the start of VI files. ³⁸

Reading strategy: Node partial reads first, workspace.fs fallback

Because `workspace.fs.readFile` reads the whole file, it’s potentially expensive for large repos or large binary files. ²³

Recommended strategy:

- If `uri.scheme === 'file'`: use Node `fs.open` + `fs.read` to read exactly 12 bytes (or 16 bytes if strict mode checks `RSRC` plus `LV??`).
- Else: use `workspace.fs.readFile(uri)` and slice just the bytes you need.

TypeScript helper: isLabviewViByMagic(uri)

The following helper is concise, safe on short files, and uses minimal IO for file URIs:

```
import * as vscode from 'vscode';
import * as fs from 'node:fs/promises';

const MAGIC_OFFSET = 8;      // 0-based
const MAGIC_LENGTH = 4;      // ASCII
const MIN_PROBE = MAGIC_OFFSET + MAGIC_LENGTH; // 12 bytes

type ViSignature = 'LVIN' | 'LVCC';

export async function isLabviewViByMagic(uri: vscode.Uri): Promise<boolean> {
    // Prefer partial read for local files
    if (uri.scheme === 'file') {
        try {
            const handle = await fs.open(uri.fsPath, 'r');
            try {
                const buf = Buffer.alloc(MIN_PROBE);
                const { bytesRead } = await handle.read(buf, 0, buf.length, 0);
                if (bytesRead < MIN_PROBE) return false;

                const magic = buf.toString('ascii', MAGIC_OFFSET, MAGIC_OFFSET +
MAGIC_LENGTH) as ViSignature;
                return magic === 'LVIN' || magic === 'LVCC';
            } finally {
                await handle.close();
            }
        } catch {
            return false;
        }
    }

    // Fallback for virtual/remote schemes: reads entire file (no range API)
    try {
        const bytes = await vscode.workspace.fs.readFile(uri); // whole file 23
        if (bytes.byteLength < MIN_PROBE) return false;

        const magic = Buffer.from(bytes).toString('ascii', MAGIC_OFFSET,
MAGIC_OFFSET + MAGIC_LENGTH) as ViSignature;
        return magic === 'LVIN' || magic === 'LVCC';
    } catch {
        return false;
    }
}
```


Unspecified detail (explicit): other LabVIEW resource files (e.g., `LVAR` for libraries, `LVSF` for projects, etc.) are not requested; treat as out of scope unless made configurable.

UI/UX design, progress reporting, and flow diagram

Context menu behavior and when-clause expression

Target UX: - Show “**VI History**” only when user right-clicks a **tracked LabVIEW VI (by magic bytes)** with ≥ 2 **modifying commits** and at least one Git repo is open.

Use a `package.json` menu contribution with `when`:

```
"contributes": {
  "commands": [
    { "command": "labviewViHistory.open", "title": "VI History" }
  ],
  "menus": {
    "explorer/context": [
      {
        "command": "labviewViHistory.open",
        "group": "3_compare",
        "when": "resourcePath in labviewViHistory.eligiblePaths &&
isWorkspaceTrusted && gitOpenRepositoryCount >= 1"
      }
    ],
    "editor/title/context": [
      {
        "command": "labviewViHistory.open",
        "group": "3_compare",
        "when": "resourcePath in labviewViHistory.eligiblePaths &&
isWorkspaceTrusted && gitOpenRepositoryCount >= 1"
      }
    ]
  }
}
```

Rationale: - `resourcePath in <contextKey>` uses VS Code's `in` operator + membership map pattern.
³⁹ - Workspace trust uses the built-in `isWorkspaceTrusted` key. ¹² - Git repo presence uses `gitOpenRepositoryCount >= 1`. ¹¹ - `editor/title/context` is supported as a menu location.
⁴ - VS Code recommends showing actions only when contextually appropriate; this aligns with the UX guidelines for context menus. ⁴⁰

History viewer design in a webview

Recommended webview UI (single panel): - Header: file name, signature (LVIN / LVCC), repo root, eligibility status - A table view with columns: commit short hash, author date, author, subject - Row actions: - **Open @ commit:** `vscode.open` on `git.toGitUri(fileUri, hash)` or CLI fallback. ⁴¹ - **Diff vs previous:** call `vscode.diff(leftUri, rightUri, title)` ⁴² - **Copy hash:** clipboard API (not covered in sources; standard VS Code) - **Generate report:** run LVCompare/LabVIEWCLI and save report as `{type}-report-{fullFilename}.html`

Webview safety: - Restrict `localResourceRoots` to extension media directory plus the report output directory inside `context.storageUri`. - Convert report file URIs using `asWebviewUri` and link them into the HTML. ⁴³ - Add CSP and minimal scripts. ²⁰

Progress bar design for large indexing and report generation

You should support three progress surfaces:

Notification / window progress via `withProgress`

VS Code's progress API supports reporting progress increments/messages, and includes a sample demonstrating cancellable notifications and `progress.report({ increment, message })`. ⁴⁴

Implementation approach: - For indexing: `window.withProgress({ location: ProgressLocation.Window, title: 'Indexing LabVIEW VI History...' }, ...)` - Report: - `percent = processed/total * 100` - message: `Scanning <repoName>: <processed>/<total> files (ETA ~ <mm:ss>)` - use a moving average over the last N file checks to estimate throughput and ETA

Status bar “discreet progress” item

VS Code UX guidelines recommend using a status bar item with a loading icon for discreet background progress. ⁴⁵

Example presentation: - Text: `$(loading~spin) VI History: 57% (1200/2100)` - Tooltip: includes repo name and ETA - Hide on completion or show a final “done” message briefly

The icon reference documents spinning animation via `~spin` for `sync`, `loading`, and `gear`. ⁴⁶

Webview progress (optional)

For long operations initiated inside the webview (e.g., “Generate report”), use `webview.postMessage({ type: 'progress', percent, message })` and render an HTML `<progress>` element with text.

Mermaid flowchart of enablement and command flow

```
flowchart TD
    A[VS Code starts / workspace opens] --> B{Workspace trusted?}
    B -- No --> B1[Hide menus via isWorkspaceTrusted] --> Z[Idle]
    B -- Yes --> C{Git repo open? gitOpenRepositoryCount >= 1}
    C -- No --> C1[Hide menus] --> Z
    C -- Yes --> D[Activate extension (onStartupFinished)]
    D --> E[Enumerate tracked files: git ls-files -z]
    E --> F[For each tracked path (concurrency-limited)]
    F --> G[Read bytes 8..11 (0-based) from file]
    G --> H{Magic is LVIN or LVCC?}
    H -- No --> F
    H -- Yes --> I[Query bounded history: git log -n 2 --format=%H --follow --<br><path>]
    I --> J{>= 2 hashes returned?}
    J -- No --> F
    J -- Yes --> K[Add absolute path to eligiblePaths map]
    K --> L[setContext('labviewViHistory.eligiblePaths', map)]
    L --> M[User right-clicks file]
    M --> N{resourcePath in eligiblePaths?}
    N -- No --> O[No VI History menu item]
    N -- Yes --> P[Show VI History menu item]
    P --> Q[Command invoked with URI]
    Q --> R[Load commits + render webview]
    R --> S{User action}
    S -->|Open@commit| T[toGitUri + vscode.open]
    S -->|Diff vs previous| U[toGitUri + vscode.diff]
    S -->|Generate report| V[Verify both blobs are VI (git show or<br>Repository.buffer)]
    V --> W[Invoke LVCompare/LabVIEWCLI, write {type}-report-{fullFilename}.html]
    W --> X[Link report into webview]
```

Sample UI mockup image/diagram instruction

Request to the downstream LLM/image generator:

Produce a VS Code-like mockup image of a “VI History” webview panel: - Top header showing: file name, repo name, signature (LVIN/LVCC), and eligibility badge. - Toolbar buttons: Refresh, Generate Diff Report, Generate Print Report. - A table with rows (hash, date, author, subject) and action buttons per row (Open, Diff, Copy Hash). - A bottom status/progress strip showing a progress bar with percent and ETA (“Generating report 42% – ETA 00:12”). - Place the panel inside a VS Code editor tab frame with typical styling.

(If generating images is not possible, render an ASCII wireframe showing the same layout.)

Report generation with LVCompare/LabVIEWCLI and mandated filename scheme

NI capabilities for comparison reports

NI documents a LabVIEW CLI operation **CreateComparisonReport** that compares two VIs and outputs an HTML/XML/Word/text report; it explicitly notes it can compare VIs “without including VI dependencies,” reducing dependency-related compare errors. ⁴⁷

Additionally, NI documents command-line usage of LVCompare/LVMerge for source control integration and that LVCompare relates to the “Compare VIs” UI. ⁴⁸

Important caveat: NI notes that comparing VIs with the same name is not supported by LVCompare.exe; a source control provider typically handles temporary renaming. ⁴⁹

This affects your extension because comparing two Git revisions will often involve the same filename. Therefore, for **head/base** comparisons you must extract blobs to *distinct temporary filenames* (e.g., `foo__base.vi` and `foo__head.vi`) before calling LVCompare, or prefer LabVIEWCLI `CreateComparisonReport` if it supports same-name inputs (capability may vary by version). Treat this as a high-risk edge case and test it.

Mandatory report filename format

All generated HTML reports must be named:

- `{type}-report-{fullFilename}.html`
- Example: `diff-report-foo.vi.html`
- The `{fullFilename}` preserves the whole filename including extensions.

Assumption (explicit): `{type}` corresponds to at least: - `diff` (semantic diff report) - optionally `print` (a “print” style report if supported by toolchain)

Verifying both revisions are VIs before running compare

Before invoking LVCompare/LabVIEWCLI, verify that *both* revisions’ blobs are VIs by reading bytes 8..11 from each blob.

Two options:

- 1) **Git API**: use `Repository.buffer(ref, path)` to fetch a `Buffer` of blob contents (fast, no process spawn). This method exists in the Git extension API. ⁵⁰
- 2) **Git CLI**: `git show <hash>:<path>` and read the first 12 bytes from stdout.

Git show for blobs returns the plain contents. ⁵¹

Security note: always set `cwd` to repo root, pass args as an array (no shell), and protect against path injection.

Where to store reports and how to render them in webview

Use `context.storageUri` to create per-workspace report directory, e.g.:

- `<storageUri>/reports/<repoId>/<safePathHash>/`
- base temp extracted VI: `base__<hash>__<fullFilename>`
- head temp extracted VI: `head__<hash>__<fullFilename>`
- report: `{type}-report-{fullFilename}.html`

This uses VS Code's recommended storage mechanism for large files. ²¹

In the webview: - Add the report output directory to `localResourceRoots`. ¹⁹ - Convert report file URI with `asWebviewUri` and provide a clickable link. ¹⁸ - Keep `localResourceRoots` as narrow as possible and add a CSP. ²⁰

Filtering PR workflows by presence of reports

The user's earlier script guidance (CI filtering by "report actually written") implies: - In automation, do not filter by `.vi` extension; filter by whether the report file exists on disk (or whether it was generated), consistent with content-based detection.

In the extension, mirror this by: - Only showing "Open report" links if the output HTML exists. - Persisting the report metadata in a small JSON alongside the report in storage.

Example TypeScript snippets for a runnable implementation

Activation and Git API acquisition

Prefer the official Git extension API pattern documented in the Git extension README, and declare `"extensionDependencies": ["vscode.git"]` to ensure activation order. ⁵²

```
import * as vscode from 'vscode';

import type { API as GitApi, GitExtension } from './git-api/git'; // copy from
vscode.git repo (see README) 53

async function getGitApi(): Promise<GitApi | undefined> {
  const ext = vscode.extensions.getExtension<GitExtension>('vscode.git');
  if (!ext) return undefined;
  const gitExt = ext.isActive ? ext.exports : await ext.activate();
  // getAPI(1) may throw if git extension is disabled 54
  return gitExt.getAPI(1);
}

export async function activate(context: vscode.ExtensionContext) {
  const git = await getGitApi();
```

```

// Initialize eligibility context map empty
await vscode.commands.executeCommand('setContext',
'labviewViHistory.eligiblePaths', {});

// Start background indexing after startup finished or immediately after
activation
context.subscriptions.push(
  vscode.commands.registerCommand('labviewViHistory.open', async (uri?:
vscode.Uri) => {
    if (!vscode.workspace.isTrusted) {
      vscode.window.showWarningMessage('Workspace is not trusted; VI History
is disabled.');
```

Eligibility indexer: git ls-files -z + magic byte + bounded git log

Use `git ls-files -z` for tracked files and safe parsing. ²⁹

Use `git log --follow` noting it works only for a single file. ³⁵

```

import * as vscode from 'vscode';
import * as cp from 'node:child_process';
import * as path from 'node:path';
import { promisify } from 'node:util';

const execFile = promisify(cp.execFile);

async function runGit(args: string[], cwd: string): Promise<{ stdout: Buffer }>
{
  const { stdout } = await execFile('git', args, {
    cwd,
    maxBuffer: 64 * 1024 * 1024,
    encoding: 'buffer' as any
  });
}
```

```

    return { stdout: stdout as Buffer };
}

async function listTrackedFiles(repoRoot: string): Promise<string[]> {
    const { stdout } = await runGit(['ls-files', '-z'], repoRoot); // 29
    return stdout
        .toString('utf8')
        .split('\0')
        .filter(Boolean);
}

async function hasAtLeastTwoModifyingCommits(repoRoot: string, relPath:
string): Promise<boolean> {
    const { stdout } = await runGit(
        ['log', '-n', '2', '--format=%H', '--follow', '--', relPath], // 28
        repoRoot
    );
    const hashes = stdout.toString('utf8').trim().split('\n').filter(Boolean);
    return hashes.length >= 2;
}

export async function buildEligibilityMap(repoRoot: string):
Promise<Record<string, boolean>> {
    const eligible: Record<string, boolean> = {};
    const files = await listTrackedFiles(repoRoot);

    // Progress: report percent/items/ETA
    const start = Date.now();
    let processed = 0;

    await vscode.window.withProgress(
        { location: vscode.ProgressLocation.Window, title: 'Indexing VI History...',
cancellable: true },
        async (progress, token) => {
            for (const rel of files) {
                if (token.isCancellationRequested) break;

                const abs = path.join(repoRoot, rel);

                // Magic byte check: LVIN/LVCC at offset 8
                const isVi = await isLabviewViByMagic(vscode.Uri.file(abs));
                if (isVi) {
                    const ok = await hasAtLeastTwoModifyingCommits(repoRoot, rel);
                    if (ok) eligible[abs] = true;
                }

                processed++;
                const percent = Math.floor((processed / files.length) * 100);

```

```

const elapsedSec = (Date.now() - start) / 1000;
const rate = processed / Math.max(elapsedSec, 0.001);
const remainingSec = (files.length - processed) / Math.max(rate, 0.001);

progress.report({
  increment: (1 / files.length) * 100,
  message: `${percent}% (${processed}/${files.length}) ETA ~$
{Math.ceil(remainingSec)}s`
});
}
}
);

return eligible;
}

```

Status bar progress item (discreet indexing)

VS Code UX guidance recommends a discreet progress status bar item for background tasks. ⁴⁵

```

import * as vscode from 'vscode';

export function createIndexerStatusBar(): vscode.StatusBarItem {
  const item =
vscode.window.createStatusBarItem(vscode.StatusBarAlignment.Left, 100);
  item.text = '$(loading~spin) VI History: indexing...'; // spinner supported 46
  item.tooltip = 'Indexing LabVIEW VIs for VI History';
  return item;
}

// Example update
function updateStatus(item: vscode.StatusBarItem, percent: number, processed:
number, total: number, etaSec: number) {
  item.text = `$(loading~spin) VI History: ${percent}% (${processed}/${total})
ETA ~${etaSec}s`;
  item.show();
}

```

Command handler: webview + vscode.diff + toGitUri

`vscode.diff` is a built-in command. ⁴²

`toGitUri` is part of the Git extension API. ⁵⁵

```

import * as vscode from 'vscode';
import type { API as GitApi } from './git-api/git';

```



```
export async function openDiff(git: GitApi, fileUri: vscode.Uri, leftHash:
string, rightHash: string) {
    const left = git.toGitUri(fileUri, leftHash); // 55
    const right = git.toGitUri(fileUri, rightHash);
    await vscode.commands.executeCommand('vscode.diff', left, right, `VI Diff: $
{leftHash.slice(0,8)} ↔ ${rightHash.slice(0,8)}`); // 42
}
```

Webview creation with localResourceRoots for report linking

Use `localResourceRoots` and `asWebviewUri`. 56

```
import * as vscode from 'vscode';

export function createHistoryWebview(
    context: vscode.ExtensionContext,
    reportDir: vscode.Uri
): vscode.WebviewPanel {
    const panel = vscode.window.createWebviewPanel(
        'labviewViHistory.panel',
        'VI History',
        vscode.ViewColumn.Active,
        {
            enableScripts: true,
            localResourceRoots: [
                vscode.Uri.joinPath(context.extensionUri, 'media'),
                reportDir
            ] // keep narrow 19
        }
    );

    const csp = panel.webview.cspSource;
    panel.webview.html = `<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8" />
    <meta http-equiv="Content-Security-Policy"
        content="default-src 'none'; img-src ${csp} https;; style-src ${csp};
script-src ${csp};">
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>VI History</title>
</head>
<body>
    <h1>VI History</h1>
    <div id="content"></div>
```

```

<script>
  const vscode = acquireVsCodeApi();
  window.addEventListener('message', (event) => {
    // update progress / table
  });
</script>
</body>
</html>`; // CSP guidance 20

  return panel;
}

```

Permissions, declarations, packaging, and publishing

Manifest declarations and trust gating

VS Code Workspace Trust onboarding supports static declarations under:

```

capabilities:
  untrustedWorkspaces:
    supported: 'limited'
    description: '...'
    restrictedConfigurations: [...]

```

The official guide describes these options and recommends using `workspace.isTrusted` + `onDidGrantWorkspaceTrust`, plus the `isWorkspaceTrusted` context key for UI gating. 57

The extension should declare: - `extensionDependencies: ["vscode.git"]` so the Git API is ready when needed. 53 - `activationEvents`: prefer `onStartupFinished` (to allow indexing without startup penalty) and `onCommand:labviewViHistory.open`. 58
(Even if you omit `onCommand` in modern VS Code, it's still acceptable; but explicit is fine.)

Packaging and publishing steps

VS Code's official Publishing Extensions guide states `vsce` is the CLI tool:

- Install: `npm install -g @vscode/vsce` 59
- Package: `vsce package` (produces `.vsix`) 60
- Publish: `vsce publish` (requires Marketplace publisher + Azure DevOps PAT with Marketplace Manage scope) 61
- Install locally for testing: `code --install-extension my-extension-0.0.1.vsix` 60

Bundling considerations: - Only bundled extensions can be used in VS Code for Web; if your extension uses `child_process` (Git CLI, LVCompare), it is inherently desktop/remote-host oriented. 62

Testing: - Use VS Code's official "Testing Extensions" guide with `@vscode/test-cli` and `@vscode/test-electron`. ⁶³ - CI can use the official Continuous Integration guidance. ⁶⁴

Testing strategy, edge cases, and implementation plan

Test cases and edge cases

Core correctness tests:

- **Magic-byte detection**

- File shorter than 12 bytes → not VI.
- File with `LVIN` at offset 8 → VI.
- File with `LVCC` at offset 8 → VI.
- File with correct magic but wrong/absent `RSRC` header → behavior depends on "strict" mode (explicitly configurable). ⁶⁵

- **Git eligibility**

- Tracked VI with 0 or 1 modifying commits → menu hidden.
- Tracked VI with ≥ 2 commits → menu shown.
- Untracked file, even if it is a VI by content → menu hidden (since `git ls-files` enumerates tracked files). ²⁹
- Renamed file:
 - With simple rename → `--follow` should find prior commits; confirm best-effort. Git documents `--follow` works only for a single file; document limitations. ³⁵

- **Repo scale/performance**

- Large tracked set (10k+ files): verify indexing remains responsive using concurrency limits, progress updates, cancellation, and caching by HEAD.
- Multi-root workspace with multiple repos: ensure eligibility maps include absolute paths across repos (no collisions) and progress text identifies which repo is being processed.

- **Submodules**

- If VS Code Git API lists submodule repos as separate repositories, index each; otherwise treat as tracked paths in root repo. Ensure that repoRoot resolution is correct per file.

- **Remote/virtual workspaces**

- In SSH/WSL/devcontainer: Node fs and child_process run on remote host (OK).

- In virtual workspaces / vscode.dev: child_process not available; the extension should either disable itself or degrade to Git API capabilities only (but comparing blobs and running LVCompare is not feasible). Bundling docs emphasize web environment constraints. ⁶⁶

- **Missing git binary / disabled Git extension**

- VS Code's docs note Git must be installed for GitHub workflows; handle "git not found" with a clear message, and/or rely on `vscode.git` and report if it's disabled. ⁶⁷

- **LVCompare same-name limitation**

- Validate that LVCompare fails if both paths have the same filename and adjust by writing blobs to unique temp names. NI notes same-name compare isn't supported by LVCompare.exe. ⁴⁹

LabVIEW 2026 Q1: 32-bit vs 64-bit LVCompare detection and selection

What NI documentation supports directly

- Default LabVIEW install locations differ by bitness:
- Windows 32-bit: `C:\Program Files (x86)\National Instruments\LabVIEW <VERSION>`
- Windows 64-bit: `C:\Program Files\National Instruments\LabVIEW <VERSION>`
- Linux: `/usr/local/natinst/LabVIEW-<VERSION>-64`
- macOS: `/Applications/National Instruments/LabVIEW <VERSION> 64-bit` ⁶⁸
- LabVIEW registry presence is documented (version stored under `HKLM\Software\National Instruments\LabVIEW`, with keys like Type/Version). ⁶⁹
- A community example explicitly details listing installed LabVIEW environments including bitness and use of WOW6432Node separation. ⁷⁰
- LVCompare source control configuration is documented and notes availability only in LabVIEW Professional Development System. ⁷¹
- LVMerge docs show the first argument can optionally be a path to `LabVIEW.exe` to select the LabVIEW version used for merge operations. ⁷²
(While this is LVMerge, it implies a pattern: prefer explicit version selection when invoking compare/merge tools.)

Recommended runtime strategy for the extension

Assumptions (explicit because NI docs don't fully specify LVCompare's bitness-selection semantics): - The extension will run on Windows/macOS/Linux, but LVCompare tooling availability and install locations vary by platform and LabVIEW edition (Professional vs Community). - The extension must work when user has LabVIEW 2026 Q1 32-bit, 64-bit, or both.

Proposed selection logic:

1) **User configuration wins** - Setting: `labviewViHistory.lvComparePath` (absolute path to LVCompare or LabVIEWCLI, depending on your chosen tool). - Setting: `labviewViHistory.labviewExePath` (absolute path to LabVIEW.exe). - Setting: `labviewViHistory.preferBitness` = `auto | x86 | x64`.

2) **Auto-discovery (Windows)** - Probe common LVCompare paths (examples commonly used in NI guidance and industry how-tos include `C:\Program Files (x86)\National Instruments\Shared\LabVIEW Compare\LVCompare.exe`, and sometimes non-x86 depending on installation). See NI integration guidance that explicitly references the x86 path for 32-bit. ⁷³

- Discover installed LabVIEW versions and bitness: - Query registry 64-bit view: `HKLM\SOFTWARE\National Instruments\LabVIEW\...` - Query registry 32-bit view: `HKLM\SOFTWARE\Wow6432Node\National Instruments\LabVIEW\...` (community example). ⁷⁴

- If registry probing is not feasible, scan default install directories for `LabVIEW.exe` using the documented defaults. ⁶⁸

3) **Auto-discovery (macOS/Linux)** - Scan documented default install locations. ⁶⁸ - Note macOS availability constraints: NI indicates LabVIEW Professional for macOS ended at 2023 Q3 and later availability is Community Edition beginning with 2025 Q3. This impacts LVCompare availability and should be documented. ⁷⁵

4) **Both architectures installed** - If the “currently running LabVIEW instance” is detectable (unspecified): optional advanced feature to inspect running processes and match LabVIEW.exe path. If not implemented, prefer: - user configured default bitness, else - x86 by default (NI generally recommends 32-bit for compatibility reasons in general guidance). ⁷⁶

Example code sketch: locating Windows LabVIEW + choosing bitness

```
import * as cp from 'node:child_process';
import { promisify } from 'node:util';
import * as fs from 'node:fs/promises';
import * as path from 'node:path';

const execFile = promisify(cp.execFile);

async function exists(p: string): Promise<boolean> {
  try { await fs.stat(p); return true; } catch { return false; }
}

// Simplified: scan default install roots (registry probing recommended for
robustness) 68
export async function findCandidateLabVIEWExeWindows(): Promise<string[]> {
  const candidates: string[] = [];
  const roots = [
    'C:\\Program Files\\National Instruments',
    'C:\\Program Files (x86)\\National Instruments'
  ];
  // Example heuristic: look for folders starting with "LabVIEW " and containing
```

```

LabVIEW.exe
for (const root of roots) {
  try {
    const entries = await fs.readdir(root, { withFileTypes: true });
    for (const ent of entries) {
      if (!ent.isDirectory()) continue;
      if (!ent.name.startsWith('LabVIEW ')) continue;
      const exe = path.join(root, ent.name, 'LabVIEW.exe');
      if (await exists(exe)) candidates.push(exe);
    }
  } catch {
    // ignore
  }
}
return candidates;
}

// Optionally query registry via reg.exe /reg:32 and /reg:64 (advanced; not
fully specified by NI docs)
export async function queryLabVIEWRegistryWindows(): Promise<string> {
  const { stdout } = await execFile('reg', [
    'query',
    'HKLM\\SOFTWARE\\National Instruments\\LabVIEW',
    '/s'
  ], { encoding: 'utf8' });
  return stdout;
}

```

Step-by-step implementation plan with milestones

Effort estimates assume a single experienced VS Code extension developer.

Milestone: Align manifest + context keys (4–6 hours)

Deliverables: - `package.json` updates: - `labviewViHistory.open` command id (or map existing) - menu contributions for `explorer/context` and optional `editor/title/context` - `when` clause exactly: `resourcePath in labviewViHistory.eligiblePaths && isWorkspaceTrusted && gitOpenRepositoryCount >= 1` - `capabilities.untrustedWorkspaces` set to `limited` with correct description and `restrictedConfigurations` if needed ⁷⁷

Acceptance: menu appears only when expected in trusted workspace with eligible file.

Milestone: Eligibility indexer robustness + caching + progress UX (8–14 hours)

Deliverables: - Cache eligibility by `(repoRoot, relPath, HEAD)` and invalidate on repo state changes. - Improve progress reporting: - percent/items/ETA in `withProgress` (increment/message) ⁴⁴ - status bar discreet progress item per UX guidelines ⁴⁵ - Add cancellation and debounce.

Acceptance: large repo indexing is cancellable, doesn't freeze UI, progress is informative.

Milestone: Webview enhancements + actions (6–10 hours)

Deliverables: - Webview with commit table, action buttons (Open, Diff, Copy hash). - Add webview progress bar area (for report generation) and message passing. - Add `localResourceRoots` and CSP hardened HTML. ⁷⁸

Acceptance: webview displays history and actions work.

Milestone: LVCompare/LabVIEWCLI report generation with required naming (12–20 hours)

Deliverables: - Report service: - Extract blobs for two commits - Verify both blobs are VI via magic bytes (either `Repository.buffer(ref,path)` ⁵⁰ or `git show`) ⁵¹ - Generate HTML report using: - LabVIEWCLI `CreateComparisonReport` ⁴⁷ and/or LVCompare - Write report to `context.storageUri` with name `{type}-report-{fullFilename}.html`. ⁷⁹ - Webview links to generated report via `asWebviewUri`. ¹⁸

Acceptance: report generation produces correctly named HTML and is viewable.

Milestone: LabVIEW 2026 Q1 bitness detection + configuration (10–18 hours)

Deliverables: - Settings: `preferBitness`, `lvComparePath`, `labviewExePath`, `labviewCliPath` - Runtime detection: - Windows registry + install scanning heuristics ⁸⁰ - macOS/Linux scanning with platform notes ⁸¹ - Handle “both installed” selection logic and user override. ⁷⁶

Acceptance: extension can auto-find tools on common installs and respects overrides.

Milestone: Test suite + CI (8–14 hours)

Deliverables: - Unit tests for magic-byte detection and git parsing (mock fs + mock git CLI). - Integration tests using `@vscode/test-electron` ⁶³ and a small fixture repo. - CI workflow using official guidance. ⁸²

Acceptance: tests run locally and in CI, covering edge cases.

Milestone: Packaging and release (4–8 hours)

Deliverables: - `vsce package` produces a VSIX. ⁶⁰ - Document installation, trust requirements, tool dependencies. - Marketplace publishing steps (PAT, publisher, vsce publish). ⁶¹

Acceptance: extension installs and runs on Windows/macOS/Linux (with tool availability documented).

Assumptions and explicitly unspecified details

- The exact set of LabVIEW file types in scope is unspecified. This report assumes **only RSRC-based VIs detectable via `LVIN` or `LVCC` at offset 8** are in scope; other LabVIEW file types (controls, libraries, etc.) are out of scope unless configured.
- `{type}` values for reports are unspecified. This report assumes at least `diff` and optionally `print`; you may map these to LVCompare/LabVIEWCLI options as supported.
- The exact LVCompare command-line switches to produce HTML reports are not fully specified by NI docs in the cited sources; therefore, the report generation recommends **LabVIEWCLI `CreateComparisonReport`** as the most explicitly documented report generator. ⁴⁷
- Determining the “currently running LabVIEW instance” to match bitness is unspecified; the report recommends user configuration first, then heuristic scanning/registry detection.
- Web extension (browser) support is not a target; because the extension shells out to git/LVCompare and uses file IO, it should be treated as desktop/remote-host oriented. ⁶²

Prioritized sources for the implementing LLM to cite

Highest priority (official):

- VS Code:
 - Contribution points / menus and command parameter inference ⁸³
 - When clause contexts (`in` , `resourcePath` , `gitOpenRepositoryCount`) ⁸⁴
 - Webview API (`localResourceRoots` , `asWebviewUri` , CSP/security) ⁸⁵
 - Workspace trust: `capabilities.untrustedWorkspaces` , `workspace.isTrusted` , `isWorkspaceTrusted` ⁸⁶
 - Progress API and sample ⁸⁷
 - Status bar UX guidelines and spinner icons ⁴⁵
 - Built-in commands (`vscode.diff` , `setContext`) ⁴²
 - Publishing extensions via vsce ⁸⁸
 - Testing extensions ⁶³
 - CI ⁶⁴
 - Git extension API README and `git.d.ts` (`toGitUri` , `Repository.log` , `Repository.buffer`) ⁸⁹
 - Proposed API limitations (`TimelineProvider`) ⁹⁰
- Git:
 - `git ls-files -z` ²⁹
 - `git log --follow` limitations ³⁵
 - `git show` blob contents ⁵¹
 - `git rev-parse` ³²
 - `gitrevisions` syntax ³¹
- NI / LabVIEW:
 - LabVIEWCLI `CreateComparisonReport` ⁴⁷
 - LVCompare / LVMerge source control configuration and command-line usage ⁹¹
 - LVCompare naming/same-name limitation note ⁴⁹
 - Default install paths by OS/bitness ⁶⁸
 - Registry location for LabVIEW version ⁶⁹
 - macOS availability note ⁷⁵
 - 32 vs 64 compatibility guidance ⁹²

Magic byte evidence sources (supplemental but required by prompt): - Ryan Pacini: RSRC header + VI signature region ³⁸ - LAVA community: first 16 bytes include RSRC and LVIN/LVCC ⁹³ - PRONOM research issue for LabVIEW VI format ³⁷

¹ ³⁸ ⁶⁵ <https://ryanpacini.com/posts/vifileformat/>
<https://ryanpacini.com/posts/vifileformat/>

2 9 10 11 39 84 <https://code.visualstudio.com/api/references/when-clause-contexts>
<https://code.visualstudio.com/api/references/when-clause-contexts>

3 12 24 57 77 86 <https://code.visualstudio.com/api/extension-guides/workspace-trust>
<https://code.visualstudio.com/api/extension-guides/workspace-trust>

4 8 13 14 83 <https://code.visualstudio.com/api/references/contribution-points>
<https://code.visualstudio.com/api/references/contribution-points>

5 21 79 87 <https://code.visualstudio.com/api/extension-capabilities/common-capabilities>
<https://code.visualstudio.com/api/extension-capabilities/common-capabilities>

6 68 81 <https://www.ni.com/docs/en-AI/bundle/labview/page/organization-of-labview.html>
<https://www.ni.com/docs/en-AI/bundle/labview/page/organization-of-labview.html>

7 44 <https://github.com/microsoft/vscode-extension-samples/blob/main/progress-sample/src/extension.ts>
<https://github.com/microsoft/vscode-extension-samples/blob/main/progress-sample/src/extension.ts>

15 16 17 58 <https://code.visualstudio.com/api/references/activation-events>
<https://code.visualstudio.com/api/references/activation-events>

18 19 20 43 56 78 85 <https://code.visualstudio.com/api/extension-guides/webview>
<https://code.visualstudio.com/api/extension-guides/webview>

22 <https://github.com/microsoft/vscode/blob/main/src/vscode-dts/vscode.proposed.timeline.d.ts>
<https://github.com/microsoft/vscode/blob/main/src/vscode-dts/vscode.proposed.timeline.d.ts>

23 <https://www.vscodetapi.com/interfaces/vscode.filesystem>
<https://www.vscodetapi.com/interfaces/vscode.filesystem>

25 52 53 89 <https://github.com/Microsoft/vscode/blob/master/extensions/git/README.md>
<https://github.com/Microsoft/vscode/blob/master/extensions/git/README.md>

26 29 <https://git-scm.com/docs/git-ls-files>
<https://git-scm.com/docs/git-ls-files>

27 33 34 41 50 54 55 <https://github.com/microsoft/vscode/blob/main/extensions/git/src/api/git.d.ts>
<https://github.com/microsoft/vscode/blob/main/extensions/git/src/api/git.d.ts>

28 35 <https://git-scm.com/docs/git-log>
<https://git-scm.com/docs/git-log>

30 51 <https://git-scm.com/docs/git-show>
<https://git-scm.com/docs/git-show>

31 <https://git-scm.com/docs/gitrevisions.html>
<https://git-scm.com/docs/gitrevisions.html>

32 <https://git-scm.com/docs/git-rev-parse>
<https://git-scm.com/docs/git-rev-parse>

36 <https://forums.ni.com/t5/LabVIEW/magic-number-for-vi/td-p/86411>
<https://forums.ni.com/t5/LabVIEW/magic-number-for-vi/td-p/86411>

37 https://github.com/digital-preservation/PRONOM_Research/issues/119
https://github.com/digital-preservation/PRONOM_Research/issues/119

40 <https://code.visualstudio.com/api/ux-guidelines/context-menus>
<https://code.visualstudio.com/api/ux-guidelines/context-menus>

42 <https://code.visualstudio.com/api/references/commands>
<https://code.visualstudio.com/api/references/commands>

45 <https://code.visualstudio.com/api/ux-guidelines/status-bar>
<https://code.visualstudio.com/api/ux-guidelines/status-bar>

46 <https://code.visualstudio.com/api/references/icons-in-labels>
<https://code.visualstudio.com/api/references/icons-in-labels>

47 <https://www.ni.com/docs/en-US/bundle/labview/page/compare-vi-cli.html>
<https://www.ni.com/docs/en-US/bundle/labview/page/compare-vi-cli.html>

48 <https://www.ni.com/docs/en-US/bundle/labview/page/comparing-vis.html>
<https://www.ni.com/docs/en-US/bundle/labview/page/comparing-vis.html>

49 <https://knowledge.ni.com/KnowledgeArticleDetails?id=kA03q000000YHwACAW&l=en-US>
<https://knowledge.ni.com/KnowledgeArticleDetails?id=kA03q000000YHwACAW&l=en-US>

59 60 61 88 <https://code.visualstudio.com/api/working-with-extensions/publishing-extension>
<https://code.visualstudio.com/api/working-with-extensions/publishing-extension>

62 66 <https://code.visualstudio.com/api/working-with-extensions/bundling-extension>
<https://code.visualstudio.com/api/working-with-extensions/bundling-extension>

63 <https://code.visualstudio.com/api/working-with-extensions/testing-extension>
<https://code.visualstudio.com/api/working-with-extensions/testing-extension>

64 82 <https://code.visualstudio.com/api/working-with-extensions/continuous-integration>
<https://code.visualstudio.com/api/working-with-extensions/continuous-integration>

67 <https://code.visualstudio.com/docs/sourcecontrol/github>
<https://code.visualstudio.com/docs/sourcecontrol/github>

69 <https://knowledge.ni.com/KnowledgeArticleDetails?id=kA00Z0000019RcJSAU&l=en-US>
<https://knowledge.ni.com/KnowledgeArticleDetails?id=kA00Z0000019RcJSAU&l=en-US>

70 74 80 <https://forums.ni.com/t5/Example-Code/List-all-LabVIEW-Environments-Installed-Windows-10/ta-p/4106097>
<https://forums.ni.com/t5/Example-Code/List-all-LabVIEW-Environments-Installed-Windows-10/ta-p/4106097>

71 91 <https://www.ni.com/docs/en-US/bundle/labview/page/configuring-source-control-with-lvcompareexe.html>
<https://www.ni.com/docs/en-US/bundle/labview/page/configuring-source-control-with-lvcompareexe.html>

72 <https://www.ni.com/docs/en-US/bundle/labview/page/configuring-source-control-with-lvmergeexe.html>
<https://www.ni.com/docs/en-US/bundle/labview/page/configuring-source-control-with-lvmergeexe.html>

73 <https://knowledge.ni.com/KnowledgeArticleDetails?id=kA0VU000000BG330AG&l=en-US>
<https://knowledge.ni.com/KnowledgeArticleDetails?id=kA0VU000000BG330AG&l=en-US>

75 <https://www.ni.com/en/support/documentation/compatibility/18/labview-and-macos-compatibility.html>
<https://www.ni.com/en/support/documentation/compatibility/18/labview-and-macos-compatibility.html>

⁷⁶ ⁹² <https://knowledge.ni.com/KnowledgeArticleDetails?id=kA00Z000000kIctSAE>
<https://knowledge.ni.com/KnowledgeArticleDetails?id=kA00Z000000kIctSAE>

⁹⁰ <https://code.visualstudio.com/api/advanced-topics/using-proposed-api>
<https://code.visualstudio.com/api/advanced-topics/using-proposed-api>

⁹³ <https://lavag.org/topic/1211-file-integrity-check-utility/>
<https://lavag.org/topic/1211-file-integrity-check-utility/>